



Assignment: Distributed Banking System

by

Name	Matric No.	Work %
Timothy Chang	<i>U2220136J</i>	33.33%
O Jing	<i>UXXXXXXX</i>	33.33%
Neo Zhi Xuan	<i>U2222293E</i>	33.33%

SC4051 Distributed Systems

NANYANG TECHNOLOGICAL UNIVERSITY

April 2026

Contents

1 Introduction & System Overview

2 Message Design & Wire Protocol

- 2.1 Packet Header Format
- 2.2 Type-Length-Value (TLV) Payload Encoding
- 2.3 Callback Push Packet
- 2.4 Cross-Language Marshalling Challenges

3 System Architecture & Design

- 3.1 Server Architecture (Go)
- 3.2 Client Architecture (C++)
- 3.3 Concurrency Model for Accounts

4 Banking Services & Custom Operations

- 4.1 Core Services (S1-S4)
- 4.2 Monitor/Callback (S4)
- 4.3 Op 5: GetBalance (Idempotent)
- 4.4 Op 6: Transfer (Non-Idempotent)

5 Fault Tolerance & Invocation Semantics

- 5.1 At-Least-Once Implementation
- 5.2 At-Most-Once Implementation
- 5.3 Loss Simulator Design
- 5.4 Experiment Design
- 5.5 Results & Discussion

6 Testing Strategy

- 6.1 Unit Tests
- 6.2 Integration Tests

7 Assumptions, Limitations & Future Work

8 Conclusion

1 Introduction & System Overview

The project implements a distributed banking application, using a custom client-server architecture that uses UDP-based inter-process communication. The system is cross-language interoperable and consists of a C++ client application and a Go-based server. These components communicate via a custom binary protocol, without Remote Procedure Call (RPC) frameworks.

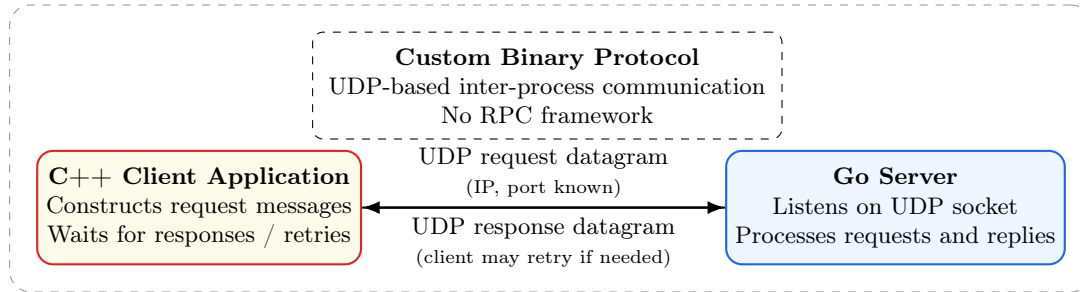


Figure 1: High-level architecture of the distributed banking system.

At a high level, the client and server communicate via UDP datagrams, on known IP addresses and ports. The client constructs request messages and waits for responses, and may retry depending on the protocol. The server listens for requests and processes each one, before returning a response.

The client is written in C++ due to the language’s ability to handle low level byte control for packet construction, as well as portable socket handling across Windows and POSIX. Meanwhile, the choice of Go for the server provides an in-depth networking library and built-in synchronisation primitives for concurrency, to allow for simple implementation of UDP sockets and concurrent client handling.

The presence of cross-language components necessitates handling of various constraints. Firstly, the data boundary and marshalling conventions for all data types must be aligned on both components. Next, it is important to standardise encodings so that both components interpret numeric bytes identically. Network byte order (big-endian) is used for all integer fields, whereas IEEE754 double format (64-bit) is used for floating-point balances.

The report analyses the unreliability of UDP, and the trade-offs between using different semantic modes to handle it, via explicit retry and duplicate-detection mechanisms.

2 Message Design & Wire Protocol

2.1 Packet Header Format

Each message sent between client and server begins with an 18-byte header, followed by a payload of variable length. It encodes information about protocol control, through these fields:

Table 1: Packet Header Layout

Field	Size	Description
Type	1 byte	Indicates whether the datagram is a request (0x00), a normal reply (0x01), or a server-initiated callback (0x02).
Semantics Flag	1 byte	Encodes invocation semantics: 0x00 for at-least-once, 0x01 for at-most-once.
Request ID	4 bytes	An identifier chosen by the client, used for duplicate detection and request-response matching.
Client IPv4 Address	4 bytes	The sender’s IPv4 address. Explicitly informs the server about the client’s identity.
Client Port	2 bytes	The UDP port number of the sender.
Status Code	2 bytes	Indicates success or error conditions in replies; non-zero values represent errors.
Content Length	4 bytes	Length (in bytes) of the payload that follows.

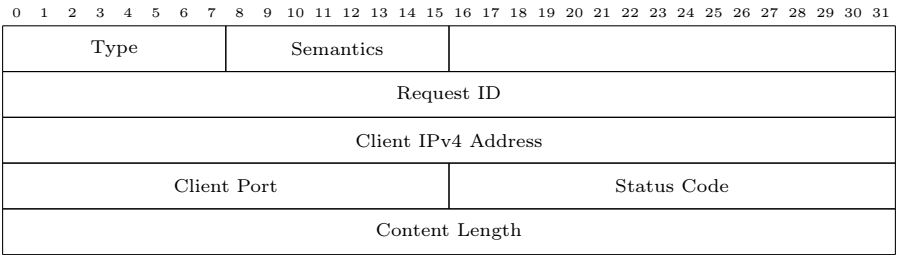


Figure 2: Byte-level representation of the packet header.

This header is inspired by traditional RPC or network protocol headers. A fixed header size allows the client and server to parse data with a consistent offset. It also allows for robust validation of datagrams that are truncated. Simultaneously, all multi-byte fields are in big-endian format, aligning the client and server’s method of interpreting numeric values.

2.2 Type-Length-Value (TLV) Payload Encoding

After the header, each message carries a TLV-encoded payload of application-specific fields. Fields are optional, and only serialised when it is present. Each field has this format:

Table 2: TLV Payload Layout

Field	Size	Description
Field ID	1 byte	Identifies the type of the field (e.g., account number, password, amount).
Length	4 bytes	Specifies the length of the Value field in bytes, encoded in big-endian format.
Value	Variable	Contains the actual data corresponding to the Field ID. The size is determined by the Length field.

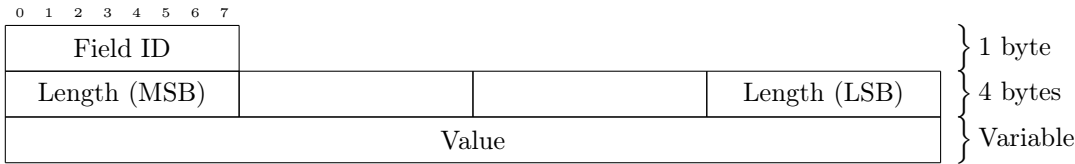


Figure 3: Type-Length-Value (TLV) encoding used in protocol payloads.

For example, the request "OpenAccount (Alice, pass1234, SGD, 1000.00)" attempts to open an account, and might include these TLV fields:

Table 3: OpenAccount Request Fields

Field Name	Length	Description
Account Holder Name	N bytes	UTF-8 encoded string representing the user’s name. The length varies depending on the number of characters.
Password	Up to 8 bytes	Password represented as raw bytes. If shorter than 8 bytes, it is zero-padded to maintain a fixed maximum length.
Currency Type	1 byte	Encoded as a predefined enumeration value representing supported currencies (e.g., SGD, USD).
Initial Balance	8 bytes	IEEE-754 double precision floating-point number encoded in big-endian format.

The client packs each numeric or string value without additional padding, and the length specifies the exact number of bytes. For example, if the user “Alice” (5 letters) chooses password “pass1234” (8 chars), currency SGD (id=1), and balance 1000.00, the TLV encoding for that request would look like:

Listing 1: TLV Encoding Example

```
01 00 00 00 05 41 6C 69 63 65      [Name field: ID=1, Len=5, Value="Alice"]
02 00 00 00 08 70 61 73 73 31 32 33 34  [Password field: ID=2, Len=8, Value="pass1234"]
```

```

03 00 00 00 01 01          [Currency field: ID=3, Len=1, Value=0x01 (SGD)]
04 00 00 00 08 40 8F 40 00 00 00 00 00 [Balance field: ID=4, Len=8, Value=1000.00]

```

The server, upon receiving this TLV payload, unpacks each field by reading the ID and length, then extracts the number of bytes as the value. It interprets the raw bytes based on the field's type - converting an 8-byte double to Go's float64 in big-endian, or treating password bytes as an ASCII string.

Meanwhile, certain fields are fixed-size. In the Go server, an 8-byte password field is enforced. This will be discussed in detail in Section 2.4.

Hence, this format handles data cleanly. The use of ID and length allows for robust parsing. While the encoding is handcrafted in code, it creates an effective platform-independent message format.

2.3 Callback Push Packet

Within the lab requirements, the server should "push" updates to all registered clients during a monitoring interval. When an account is opened, closed, deposited, or withdrawn, the server triggers a callback packet to the client. It uses a different packet format:

Table 4: Callback Packet Layout

Field Name	Size	Description
MsgType	1 byte	Always set to 0x02 to differentiate callback packets from normal replies.
Service ID	1 byte	Indicates the operation to trigger (e.g., OpenAccount).
Account Number	4 bytes	Integer account number (big-endian).
Name Length	4 bytes	Length of the account holder's name in bytes.
Name	Variable	UTF-8 encoded string representing the account holder's name.
Currency Type	1 byte	The account's currency code as an enumeration.
Balance	8 bytes	IEEE-754 double precision floating-point value (big-endian) representing the updated balance.

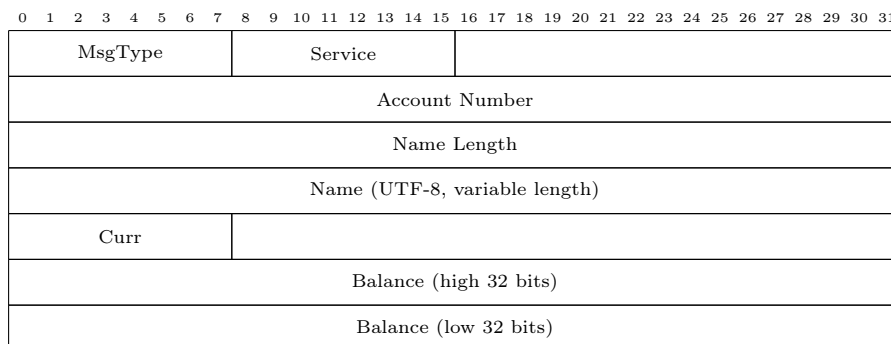


Figure 4: Byte-level representation of the callback payload.

For example, if account 12345 ("Bob") with currency USD (0x00) now has balance 250.75, the server might send:

Listing 2: Callback Packet Example

```

00 00 30 39          [Account#=12345 (big-endian)]
00 00 00 03 42 6F 62 [NameLen=3, Name="Bob"]
00                  [Currency=0x00 (USD)]
40 6F 40 00 00 00 00 00 [Balance=250.75 (IEEE-754)]

```

The client's UDP socket listens for these packets and, upon receipt, displays the updated account information to the user.

The benefits of having a fixed, known structure is that it avoids the overhead that TLV packets face with self-describing fields. TLV packets require 5 additional bytes for each field to store FieldID and Length, making it slower to encode, decode, and process.

2.4 Cross-Language Marshalling Challenges

Implementing the client and server in different languages always leads to challenges in data coordination, due to the differences in how each language defines its data types and mechanisms. The first challenge was designing the password field. The Go server defines passwords as a fixed 8-byte array, whereas the C++ client uses ‘std::string’. To overcome this, the protocol’s TLC password field was fixed to 8 bytes on the wire. The client would ensure that the string is 8 bytes, by truncating it, or padding it with zeros.

The second issue is the different representation of floating-point values in memory across both languages, where C++ uses ‘double’ and Go uses ‘float64’. While both languages use IEEE-754 64-bit binary representation for these numbers, they could differ in endianness, and lead to incorrect decoding of raw bytes. Hence, for all floating-point transmissions, values are converted into IEEE-754 format encoded in big-endian order before sending, and recipients would reverse the process. This ensures the same byte sequence is interpreted reliably and correctly.

3 System Architecture & Design

3.1 Server Architecture (Go)

The Go server is implemented with a layered design.



Figure 5: Go server architecture (high-level).

Firstly, main function sets up a UDP socket and constructs the server components.

Next, the dispatcher reads incoming UDP packets, unmarshals the payload and header, before routing each request to an appropriate service handler. Its behaviour depends on the chosen invocation semantics: for at-least-once mode, it forwards each request to the handler, while in at-most-once mode, it checks the request against a history table to eliminate duplicate requests.

Thirdly, the handler receives a request from the dispatcher, and chooses a service to trigger. It contains the application logic for services like opening, closing, depositing and withdrawing from an account. It decodes the TLV fields from the payload and performs validation, before invoking the appropriate service function, and encoding a TLV response.

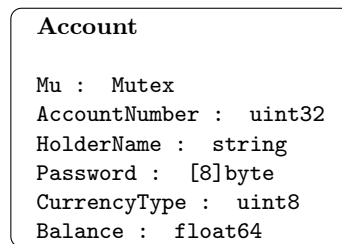


Figure 6: Memory store account structure.

Lastly, the banking service and memory store layers handle the data operations. The service layer provides operations like OpenAccount that directly manipulate the storage. The memory store maintains a map of accounts, and uses synchronisation primitives to ensure that concurrent access to data is safe and does not lead to race conditions.

Separately, the monitoring feature is implemented within the banking service with a publish-subscribe mechanism. It stores a map of registered clients with their defined expiration times, and handles the goroutines for dispatching account updates using callback packets.

By separating the logic into layers, we isolate concerns cleanly. For instance, semantics logic is handled in the dispatcher, and does not clutter banking service logic. Protocol encoding is handled only once in the dispatcher and handler. At the same time, this allows dependency injection in tests, by allowing developers to mock the UDP socket at the top level.

The architecture is wired by a BuildHandler function that exposes the banking service and monitor manager to all handlers. This ensures that the dispatcher can make calls to the handler without having to know the underlying banking or monitoring details, hence decoupling these components.

3.2 Client Architecture (C++)

The C++ client is implemented with a three-layer architecture.



Figure 7: C++ client layered architecture.

When sending a request from the application layer, the client runs an interactive looping Command Line Interface (CLI) that takes a command from the user corresponding to banking operations. It communicates this request to the protocol layer. The protocol layer constructs a request message by populating the 18-byte header with the chosen invocation semantics flag and a fresh or zeroed request ID. It then converts numbers into big-endian format before encoding arguments into a TLV payload for cross-platform consistency. This layer also manages the decoding of replies and callbacks into structured data. Finally, the network layer abstracts UDP communication by providing send and receive operations over sockets.

When the client receives a response, the response takes the reverse path. It is received at the network layer. Then, the protocol layer validates headers and parses the payload, and thereafter the application layer presents the results or callback updates to the user. This design ensures that user logic, message formatting, and network communication are cleanly decoupled and easy to maintain.

At the same time, dependency injection was used to provide the client with access to various services. The client was able to access interfaces like `BaseSocket` and `BaseCommandEncoder` during its lifecycle. This allows the system to mock components for unit testing, and reduces coupling between the interface and networking/marshalling internals.

3.3 Concurrency Model for Accounts

The system was designed to allow multiple clients to operate concurrently on the server, and requires various synchronisation strategies to align this.

The in-memory account map, where account numbers were mapped to account data, is protected by a global read-write mutex which synchronises account lifecycle operations like `CreateAccount` and `GetAccount`. While multiple read queries can be run under a shared read lock, write-based operations utilise an exclusive write lock.

Each account's state is protected separately using mutexes that belongs to the account itself. After accessing the specific account from the store, the account's mutex is acquired before read or mutation operations, and only unlocked when completed. This design allows clients to concurrently operate on different accounts while ensuring correctness on individual accounts.

4 Banking Services & Custom Operations

4.1 Core Services (S1–S4)

4.2 Monitor/Callback (S4)

4.3 Op 5: GetBalance (Idempotent)

4.4 Op 6: Transfer (Non-Idempotent)

5 Fault Tolerance & Invocation Semantics

5.1 At-Least-Once Implementation

5.2 At-Most-Once Implementation

5.3 Loss Simulator Design

5.4 Experiment Design

5.5 Results & Discussion

6 Testing Strategy

6.1 Unit Tests

6.2 Integration Tests

7 Assumptions, Limitations & Future Work

8 Conclusion